Bonjour Claude,

J'ai un projet de **développement web** et j'aimerais que tu m'aides à le réaliser étape par étape, de manière simple, détaillée et directement exploitable.

À partir du cahier des charges (version 2.0, web) et de la présentation du projet **MediScan AI**, je souhaite développer une **application web de santé intelligente** destinée principalement aux patients et aux médecins.

Thème du projet : création d'une application web avec une interface patient et une interface médecin, qui a pour objectif de traduire les données personnelles des patients (anamnèse) en données exploitables pour les médecins via l'intelligence artificielle (IA), complétée par la reconnaissance d'image des médicaments.

1. Technologies à utiliser

Je souhaite utiliser les technologies suivantes :

- **Angular** (TypeScript) pour le développement du frontend web, avec **VS Code** comme éditeur ;
- **Angular Material** (ou Bootstrap) pour les composants d'interface et le design responsive ;
- **Spring Boot avec Java** pour développer le backend et l'API REST ;
- **MySQL** pour la base de données ;
- **Postman** pour tester les différentes API ;
- **Firebase Cloud Messaging uniquement pour la partie notifications** (notifications Web Push dans le navigateur), notamment pour les rappels et les notifications destinés aux utilisateurs ;
- Une solution d'**intelligence artificielle / OCR** pour analyser les images des médicaments et extraire automatiquement leurs informations.

L'architecture doit être organisée de manière claire avec une séparation entre le frontend Angular, le backend Spring Boot, la base de données MySQL et le service IA.

2. Objectif de l'application

L'application **MediScan AI** doit permettre de faciliter la collecte et la gestion des informations médicales du patient et d'aider le médecin à consulter ces informations de manière structurée, depuis un simple navigateur (ordinateur, tablette ou smartphone).

L'application doit comporter deux interfaces principales :

- **Interface Patient**
- **Interface Médecin**

Les deux interfaces communiquent avec le backend Spring Boot via des API REST. Les données sont enregistrées dans MySQL. Le cahier des charges prévoit notamment l'authentification, l'anamnèse, le scan des médicaments, les rappels, le partage sécurisé avec un médecin et le dashboard médical.

3. Fonctionnalités de l'interface Patient

L'application web doit permettre au patient de :

1. Créer un compte ;

2. Se connecter ;
3. Gérer son profil ;
4. Remplir son **anamnèse médicale** à travers un formulaire guidé (formulaire multi-étapes) ;
5. Ajouter ses antécédents médicaux ;
6. Ajouter ses allergies ;
7. Ajouter ses pathologies chroniques ;
8. Ajouter ses traitements en cours ;
9. Importer la photo d'un médicament depuis son ordinateur ou son téléphone, ou la capturer directement avec la **webcam** du navigateur ;
10. Envoyer l'image au backend ;
11. Utiliser l'IA/OCR pour extraire automatiquement les informations du médicament ;
12. Afficher les informations extraites : nom du médicament, DCI, dosage, forme, posologie, contre-indications, interactions éventuelles, numéro de lot, date d'expiration ;
13. Consulter l'historique des médicaments scannés ;
14. Consulter son historique médical ;
15. Configurer des rappels pour la prise des médicaments ;
16. Recevoir les notifications dans le navigateur via Firebase (Web Push) ;
17. Partager certaines informations médicales avec un médecin après avoir donné son consentement explicite.

4. Fonctionnalités de l'interface Médecin

L'interface Médecin doit permettre de :

1. Créer un compte médecin ;
2. Se connecter ;
3. Consulter son dashboard ;
4. Voir la liste de ses patients ;
5. Consulter la fiche d'un patient ;
6. Consulter l'anamnèse du patient ;
7. Consulter l'historique des médicaments ;
8. Consulter les médicaments détectés par l'IA ;
9. Visualiser les alertes ;
10. Détecter les interactions ou contre-indications basiques ;
11. Consulter l'historique complet du patient ;
12. Accéder aux données uniquement lorsque le patient a donné son consentement.

Le dashboard doit présenter les informations importantes de manière simple et claire (cartes, tableaux, graphiques), notamment les patients, les analyses et les alertes.

5. Fonctionnalité principale : Scan intelligent d'un médicament

Je souhaite que le fonctionnement soit organisé comme suit :

Patient → Navigateur (import de fichier ou webcam) → Image du médicament → Backend Spring Boot → IA/OCR → Extraction des informations → Vérification → MySQL → Résultat affiché dans Angular

Lorsqu'un patient importe ou capture la photo du packaging d'un médicament :

1. Angular récupère l'image (champ fichier ou flux webcam via `getUserMedia`) et affiche un aperçu ;
2. L'application envoie l'image au backend Spring Boot (requête `multipart/form-data` avec le token JWT) ;
3. Le backend transmet l'image au service IA/OCR ;
4. L'IA analyse l'image ;
5. Les informations du médicament sont extraites ;
6. Les informations sont vérifiées avec une base de médicaments ;
7. Le résultat est enregistré dans MySQL ;
8. Le backend retourne les informations à l'application Angular au format JSON ;
9. L'application affiche les résultats au patient ;
10. Une alerte doit être affichée lorsqu'une interaction ou une contre-indication est détectée (et une notification Web Push peut être envoyée).

6. Backend Spring Boot

Je souhaite que le backend soit développé proprement avec une architecture en couches : Controller, Service, Repository, Entity, DTO, Security, Configuration (dont la configuration **CORS** autorisant l'origine du frontend Angular).

Il doit notamment contenir les API pour : authentification, utilisateurs, patients, médecins, profils, anamnèse, médicaments, scan de médicaments, analyse IA, alertes, rappels, notifications (enregistrement des tokens FCM du navigateur), partage des données et consentement.

L'authentification doit utiliser **JWT** et les mots de passe doivent être sécurisés avec **BCrypt**, conformément au cahier des charges.

7. Frontend Angular

Je souhaite que le frontend Angular soit organisé proprement :

- **Modules ou composants standalone** par fonctionnalité (auth, patient, médecin, shared) ;
- **Services** Angular pour les appels HTTP (HttpClient) vers l'API ;
- **Intercepteur HTTP** qui ajoute automatiquement le token JWT dans l'en-tête Authorization ;
- **Guards** de routes pour protéger les pages selon le rôle (PATIENT / MEDECIN) ;

- **Formulaires réactifs** avec validation pour l'inscription, l'anamnèse et les rappels ;
- **Modèles TypeScript** (interfaces) correspondant aux DTO du backend ;
- Fichiers **environment** pour l'URL de l'API et la configuration Firebase ;
- Design **responsive** utilisable sur ordinateur, tablette et smartphone.

8. Base de données MySQL

Je souhaite une base de données MySQL correctement structurée avec les principales tables nécessaires, par exemple : users, patients, medecins, anamneses, medicaments, scans, alertes, rappels, notifications, partages, consentements, device_tokens (tokens FCM des navigateurs).

Il faut créer les relations entre les tables et fournir le script SQL permettant de créer la base de données.

9. Firebase

Important : je souhaite utiliser Firebase **uniquement pour les notifications**, via **Firebase Cloud Messaging (Web Push)**.

Je ne souhaite pas utiliser Firebase comme base de données principale ni remplacer MySQL. MySQL doit rester la base de données principale du projet. Je n'utilise pas non plus Firebase Hosting ni Firebase Authentication.

Côté navigateur, il faut configurer le **service worker** de Firebase Messaging, demander l'autorisation des notifications à l'utilisateur et envoyer le token obtenu au backend. Côté Spring Boot, il faut utiliser le **SDK Firebase Admin** pour envoyer les notifications, par exemple :

- rappel de prise d'un médicament (déclenché par une tâche planifiée `@Scheduled`) ;
- notification d'une nouvelle alerte ;
- notification lorsqu'un médecin consulte ou reçoit les données autorisées ;
- autres notifications utiles de l'application.

10. Sécurité

Le projet doit prendre en compte la confidentialité des données médicales. Il faut notamment prévoir :

- authentification sécurisée ;
- JWT ;
- BCrypt ;
- contrôle des accès Patient/Médecin côté backend (Spring Security) et côté frontend (guards Angular) ;
- consentement explicite du patient ;
- protection des données médicales ;
- validation des données envoyées par le navigateur (côté Angular et côté Spring Boot) ;
- configuration CORS limitée à l'origine du frontend ;

- protection contre les failles web courantes (XSS, CSRF selon le mode de stockage du token) ;
- communication sécurisée (HTTPS) entre le navigateur et Spring Boot.

Le cahier des charges insiste également sur la sécurité, la confidentialité, le consentement et la traçabilité.

11. Développement étape par étape

Je souhaite que tu me guides étape par étape et en détail, en commençant par la création complète du projet. Je souhaite suivre cet ordre :

1. Préparation de l'environnement de développement (JDK, Node.js, Angular CLI, MySQL, IntelliJ, VS Code, Postman)
2. Création du projet Spring Boot
3. Configuration MySQL
4. Création des entités et relations JPA
5. Authentification JWT
6. Création des API REST (et configuration CORS)
7. Test du backend avec Postman
8. Création du projet Angular (Angular CLI)
9. Création de l'interface Patient
10. Création de l'interface Médecin
11. Connexion Angular ↔ Spring Boot (services, intercepteur JWT, guards)
12. Création du formulaire d'anamnèse
13. Intégration de l'import d'image / webcam et du scan médicament
14. Intégration de l'IA/OCR
15. Enregistrement des résultats dans MySQL
16. Création des alertes
17. Intégration de Firebase Cloud Messaging uniquement pour les notifications web
18. Création du dashboard Médecin
19. Gestion du partage et du consentement
20. Tests complets de l'application
21. Correction des erreurs
22. Préparation de la version finale du projet (build de production Angular, déploiement).

12. Structure du projet

Je souhaite également que tu me donnes une structure complète et propre des dossiers, par exemple :

```
mediscan-ai/
├── mediscan-ai-backend/      (Spring Boot)
├── mediscan-ai-frontend/     (Angular)
├── database/                 (scripts SQL)
├── postman/                  (collection Postman)
└── documentation/
```

Explique-moi où placer chaque fichier, chaque classe Java et chaque composant ou service Angular.

13. Code complet

Pour chaque étape, je souhaite recevoir :

- le code complet ;
- le nom exact du fichier ;
- le chemin du fichier ;
- les commandes à exécuter (Maven, npm, ng) ;
- les dépendances à installer ;
- les configurations nécessaires ;
- les explications simples ;
- les tests à effectuer ;
- les erreurs possibles et leurs corrections (dont les erreurs CORS courantes).

Évite de me donner uniquement des extraits de code incomplets. Lorsque plusieurs fichiers sont nécessaires, donne-moi les fichiers complets afin que je puisse les copier directement dans mon projet.

14. Projet ZIP

À la fin du développement, je souhaite obtenir un projet complet prêt à être exécuté, idéalement sous forme d'un fichier ZIP, contenant :

- le backend Spring Boot ;
- le projet Angular ;
- les scripts MySQL ;
- la collection Postman ;
- les fichiers de configuration nécessaires (dont le service worker Firebase) ;
- la documentation d'installation et d'exécution.

Je souhaite pouvoir décompresser le ZIP, ouvrir les projets dans les outils correspondants, configurer mes paramètres locaux, puis lancer le backend (`mvn spring-boot:run`) et le frontend (`npm install` puis `ng serve`).

15. Résultat final attendu

Le résultat final doit être un MVP fonctionnel de MediScan AI, accessible depuis un navigateur, permettant de réaliser le parcours suivant :

Inscription → Connexion → Profil Patient → Anamnèse → Import/capture d'un médicament → Analyse IA/OCR → Vérification → Enregistrement MySQL → Affichage du résultat → Alertes → Rappels/Notifications Firebase (Web Push) → Partage sécurisé avec le médecin → Dashboard Médecin.

Le projet doit rester adapté à un contexte académique/MVP, avec une architecture claire, simple à comprendre, maintenable et évolutive.

Merci de commencer par l'étape 1, puis de continuer progressivement jusqu'à obtenir le projet complet.